

Tutorial completo

Validação de banco de dados com Python, Pytest e SQLite

Guia passo a passo do projeto no PyCharm, incluindo execução, interpretação dos resultados e correção dos erros encontrados.

Projeto: Automacao-de-Testes-com-Playwright-Python-e-Pytest
Banco: stores/loja.db

1. Objetivo

Este guia explica como preparar o projeto, conferir o banco SQLite, executar um teste individual ou todos os testes, interpretar o resultado e resolver os problemas que apareceram durante a prática.

Resultado já confirmado: o teste `test_validar_lista_produtos` passou depois da remoção dos produtos duplicados. A saída foi 1 passed. Isso confirma esse teste individual, não os quatro testes juntos.

2. Estrutura do projeto

```
Automacao-de-Testes-com-Playwright-Python-e-Pytest/  
■■■■ .venv/  
■■■■ PageObjects/  
■   ■■■■ Page/  
■       ■■■■ utilities/  
■           ■■■■ bancosDeDados.py  
■■■■ ValidacaoBanco/  
■   ■■■■ test_validar_banco.py  
■■■■ stores/  
■■■■ loja.db
```

A pasta principal do projeto é a raiz. O banco fica em `stores/loja.db`. O utilitário fica em `PageObjects/Page/utilities/bancosDeDados.py` e os testes em `ValidacaoBanco/test_validar_banco.py`.

3. Preparar o ambiente

3.1 Abrir o terminal na raiz

No PyCharm, abra Terminal. O caminho deve terminar na pasta principal do projeto, não em `PageObjects`. Se estiver em `PageObjects`, execute:

```
cd ..
```

3.2 Conferir Python, SQLAlchemy e banco

```
python --version  
python -m pip show SQLAlchemy  
Test-Path .\stores\loja.db
```

`Test-Path` deve retornar `True` se o arquivo estiver nesse local. Use o interpretador `.venv` do projeto, como aparece na execução do PyCharm.

4. Utilitário `bancosDeDados.py`

O arquivo precisa ter as funções importadas pelo teste. Abaixo está a versão usada no tutorial. Confira se seu arquivo contém estas funções e mantenha o caminho resolvido a partir da raiz do projeto.

```
from pathlib import Path  
from sqlalchemy import create_engine, text  
  
def montar_connection_string(tipo_banco, database):  
    if tipo_banco.lower() == "sqlite":  
        raiz_projeto = Path(__file__).resolve().parents[3]  
        caminho_banco = Path(database)  
        if not caminho_banco.is_absolute():  
            caminho_banco = raiz_projeto / caminho_banco  
        return f"sqlite:///{{caminho_banco.as_posix()}}"  
        raise ValueError("Tipo de banco não suportado")  
  
def criar_engine(connection_string):  
    return create_engine(connection_string)  
  
def executar_query(banco, query, parametros=None):  
    with banco.connect() as conexao:  
        resultado = conexao.execute(text(query), parametros or {})  
        return resultado.fetchall()  
  
def registro_existe(banco, query, parametros=None):  
    resultado = executar_query(banco, query, parametros)  
    assert len(resultado) > 0, "Nenhum registro foi encontrado"  
  
def contar_registros(banco, query, quantidade_esperada, parametros=None):  
    resultado = executar_query(banco, query, parametros)
```

```

    assert len(resultado) == quantidade_esperada, (
        f"Esperado: {quantidade_esperada}; encontrado: {len(resultado)}"
    )

def validar_valor_coluna(banco, query, coluna, valor_esperado, parametros=None):
    resultado = executar_query(banco, query, parametros)
    assert len(resultado) > 0, "Nenhum registro foi encontrado"
    nomes_colunas = resultado[0]._mapping.keys()
    assert coluna in nomes_colunas, f"Coluna '{coluna}' não encontrada"
    valor_obtido = resultado[0]._mapping[coluna]
    assert valor_obtido == valor_esperado, (
        f"Esperado: {valor_esperado}; encontrado: {valor_obtido}"
    )

def validar_resultado_esperado(banco, query, resultado_esperado):
    resultado = executar_query(banco, query)
    resultado_obtido = [list(linha) for linha in resultado]
    assert resultado_obtido == resultado_esperado, (
        f"Resultado esperado: {resultado_esperado}\\n"
        f"Resultado obtido: {resultado_obtido}"
    )

```

Atenção ao caminho: como o teste passa `database='stores/loja.db'`, o utilitário não deve acrescentar outra pasta `stores`. Isso evita formar `stores/stores/loja.db`.

5. Arquivo de testes

O arquivo `ValidacaoBanco/test_validar_banco.py` deve importar as funções do utilitário e conter quatro funções de teste. O código abaixo reúne a versão compartilhada:

```

from PageObjects.Page.utilities.bancosDeDados import (
    criar_engine, montar_connection_string, registro_existe,
    contar_registros, validar_valor_coluna, validar_resultado_esperado,
)

def criar_banco():
    url = montar_connection_string("sqlite", database="stores/loja.db")
    return criar_engine(url)

def test_usuario_existe():
    banco = criar_banco()
    query = "SELECT * FROM usuarios WHERE email = :email"
    registro_existe(banco, query, {"email": "joao@teste.com"})

def test_contar_usuarios_ativos():
    banco = criar_banco()
    query = "SELECT * FROM usuarios WHERE ativo = 1"
    contar_registros(banco, query, quantidade_esperada=2)

def test_validar_preco_produto():
    banco = criar_banco()
    query = "SELECT nome, preco FROM produtos WHERE nome = :nome"
    validar_valor_coluna(
        banco, query, coluna="preco", valor_esperado=199.90,
        parametros={"nome": "Calca Jeans"}
    )

def test_validar_lista_produtos():
    banco = criar_banco()
    query = "SELECT nome, estoque FROM produtos ORDER BY id"
    produtos_esperados = [
        ["Camiseta Preta", 50],
        ["Calca Jeans", 20],
        ["Tenis Esportivo", 10],
    ]
    validar_resultado_esperado(banco, query, produtos_esperados)

```

6. O que cada teste verifica

test_usuario_existe

Procura o usuário `joao@teste.com`. Se nenhuma linha for encontrada, o teste falha.

test_contar_usuarios_ativos

Conta as linhas com `ativo = 1` e espera exatamente duas.

test_validar_preco_produto

Busca Calça Jeans e compara o preço com 199.90.

test_validar_lista_produtos

Compara os nomes e estoques com os três produtos esperados, ordenados pelo id.

7. Executar os testes

7.1 Um teste no PyCharm

Abra test_validar_banco.py. Clique com o botão direito na função desejada, por exemplo test_validar_lista_produtos, e escolha Run 'test_validar_lista_produtos'.

7.2 Todos os testes no PyCharm

Clique com o botão direito no arquivo test_validar_banco.py e escolha Run correspondente ao arquivo/pytest. Isso deve executar as quatro funções.

7.3 Pelo terminal

Na raiz do projeto, execute:

```
python -m pytest -v ValidacaoBanco/test_validar_banco.py
```

Para executar somente a lista de produtos:

```
python -m pytest -v ValidacaoBanco/test_validar_banco.py::test_validar_lista_produtos
```

8. Entender a saída

Saída	Significado
PASSED / passed	O teste executou e a validação foi aprovada.
FAILED / failed	O teste executou, mas o resultado não correspondeu ao esperado.
ERROR na coleta/importação	O pytest não conseguiu preparar/importar o teste.
1 passed	Um teste foi executado e passou.
4 passed	Quatro testes foram executados e passaram.
exit code 0	O processo terminou com sucesso.

9. Problemas vistos e soluções

ImportError: registro_existe

O teste importava uma função ausente em bancosDeDados.py. Solução: incluir a função e as demais validações no utilitário, ou corrigir o import para o nome real.

unable to open database file

O terminal estava em PageObjects ou o caminho estava errado. Volte à raiz com cd .. e confirme Test-Path .\stores\loja.db. Evite duplicar stores no caminho.

Lista esperava 3 produtos, mas encontrou 6

O banco tinha duplicados: IDs 1 a 3 e novamente 4 a 6. A consulta mostrou duas cópias dos três produtos. Após a remoção dos registros 4, 5 e 6, o teste passou.

Cuidado ao apagar registros

Faça backup e confirme que os IDs são duplicados e não são usados por pedidos ou outros testes. DELETE altera o banco.

```
Copy-Item .\stores\loja.db .\stores\loja_backup.db
python -c "import sqlite3; c=sqlite3.connect('stores/loja.db'); c.execute('DELETE FROM produtos WHERE id IN (4,5,6)'); c.commit"
```

Execute esse comando somente no terminal, na raiz do projeto, e apenas quando tiver certeza de que os registros devem ser removidos. Não é necessário repeti-lo se a limpeza já foi feita.

UNIQUE constraint failed

Pode acontecer ao inserir novamente um valor que precisa ser único, como e-mail. Confira os registros antes de repetir o script de carga de dados.

10. Checklist antes de finalizar

- Terminal na raiz do projeto.
- Interpretador .venv correto selecionado.
- Arquivo stores/loja.db existe.
- Utilitário tem todas as funções importadas.
- Caminho não duplica stores.
- Dados reais correspondem aos valores esperados.
- Teste individual executado.
- Arquivo completo executado para confirmar os quatro testes.

11. Resultado confirmado

O teste `test_validar_lista_produtos` passou após a correção dos duplicados: 1 passed in 0.21s, exit code 0. Para confirmar todos os testes, execute o arquivo inteiro e confira se aparece 4 passed.